Gated Graph Recurrent Neural Networks

Piotr Lipinski

Outline

1. RNN Background
2. Graph Signal Processing
3. Graph Recurrent Neural Networks
4. Gated Graph Recurrent Neural Networks

RNN Background

Classical RNN

hidden state evolution:

$$z_t = \sigma(Ax_t + Bz_{t-1})$$

output:

$$y_t = \rho(Cz_t)$$

where:

- $x_t =$ input
- $z_t =$ hidden state
- $y_t =$ output
- $A, B, C =$ parameter matrices; $\sigma, \rho =$ nonlinearities (pointwise)

Limitations of RNNs

Problem:

Graph processes contain both

time structure

and

graph structure.

The graph topology is ignored by a standard RNN.

Graph Signal Processing

Graph Signals

Graph:

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{W})$$

where $\mathcal{W}$ denotes a weight function on edges.

Signal:

$$x \in \mathbb{R}^N$$

where node i stores value x_i.

Graph Shift Operator

Graph structure encoded by:

$$S \in \mathbb{R}^{N \times N}$$

Examples:

- adjacency matrix
- Laplacian
- random-walk matrix

Local diffusion:

$$Sx$$

Each node aggregates information from neighbors.

Diffusion Interpretation

Repeated shifts:

$$S^2x, \quad S^3x, \ldots$$

Interpretation:

- Sx = 1-hop information
- S^2x = 2-hop information
- S^kx = k-hop information

Graph propagation becomes localized and distributed.

Graph Convolution

Linear Shift Invariant Graph Filter:

$$A(S)x = \sum_{k=0}^{K-1} a_k S^k x$$

Properties:

- local computation
- K-hop receptive field
- parameter sharing
- graph analogue of convolution

Graph Processes

Graph process:

$$\{x_t\}_{t \in \mathbb{N}}$$

Remarks:

- graph signal evolving over time
- fixed topology
- changing node values

It needs:

$$\text{Graph structure} + \text{Temporal memory}$$

Graph Recurrent Neural Networks

Main Idea of GRNN

Replace dense matrices by graph filters.

Classical RNN:

$$Ax_t + Bz_{t-1}$$

GRNN:

$$A(S)x_t + B(S)z_{t-1}$$

Result:

- spatial modeling
- temporal modeling

GRNN State Update

hidden state evolution:

$$z_t = \sigma\left(A(S)x_t + B(S)z_{t-1}\right)$$

it addresses:

- input graph filtering
- recurrent graph filtering
- nonlinear update

GRNN Filters

Input filter:

$$A(S) = \sum_{k=0}^{K-1} a_k S^k$$

State filter:

$$B(S) = \sum_{k=0}^{K-1} b_k S^k$$

Only K coefficients per filter.

Parameter count does not grow with graph size.

$BS(S)$ **may lead to oversmoothing! and other problems!**

Gated Graph Recurrent Neural Networks

Why Gating?

Problem:

- vanishing gradients
- exploding gradients
- long temporal dependencies
- oversmoothing and oversquashing

Generic Gated GRNN (GGRNN)

hidden state evolution:

$$Z_t = \sigma\Big(\hat{Q}\{A_S(X_t)\} + \check{Q}\{B_S(Z_{t-1})\}\Big)$$

where
$\hat{Q} : \mathbb{R}^{NxH} \to \mathbb{R}^{NxH}$ is the input gate operator, and
$\check{Q} : \mathbb{R}^{NxH} \to \mathbb{R}^{NxH}$ is the forget gate operator.
Depending on the gating strategy (time gating, node gating, edge gating, etc.) they may have different forms.

Gate Generation

input gate state:

$$\hat{Z}_t = \hat{\sigma}(\hat{A}_S(X_t) + \hat{B}_S(\hat{Z}_{t-1})) \in \mathbb{R}^{N \times \hat{H}}$$

forget gate state:

$$\check{Z}_t = \check{\sigma}(\check{A}_S(X_t) + \check{B}_S(\check{Z}_{t-1})) \in \mathbb{R}^{N \times \check{H}}$$

Gates are themselves generated by GRNNs.

Time Gating GRNN (t-GGRNN)

Input gate:

$$\hat{Q}\{A_S(X_t)\} = \hat{q}_t A_S(X_t)$$

Forget gate:

$$\check{Q}\{B_S(Z_t)\} = \check{q}_t B_S(Z_t)$$

Global scalar gates.

Time Gate Parameters

Gate values:

$$\hat{q}_t = \text{sigmoid}(\hat{c}^T \operatorname{vec}(\hat{Z}_t))$$

$$\check{q}_t = \text{sigmoid}(\check{c}^T \operatorname{vec}(\check{Z}_t))$$

Interpretation:

- similar to LSTM
- controls temporal memory
- same gate for all nodes

Node Gating GRNN (n-GGRNN)

Input gate:

$$\hat{Q}\{A_S(X_t)\} = diag(\hat{q})A_S(X_t))$$

Forget gate:

$$\check{Q}\{B_S(Z_t)\} = \check{d}iag(\check{q}_t)B_S(Z_t)$$

$$\hat{q}_t = \text{sigmoid}(\hat{C}_S\,(\hat{Z}_t))$$

$$\check{q}_t = \text{sigmoid}(\check{C}_S\,(\check{Z}_t))$$

Node Gating GRNN (n-GGRNN)

Motivation:

Different graph regions may exhibit different dynamics.

Idea:

- one gate per node
- node-dependent memory
- spatially adaptive recurrence

Edge Gating

Motivation:

Some edges should be active, others should be suppressed.

Idea:

- gate every edge
- adaptive information flow
- graph-dependent attention mechanism